An Odd-Support Obstruction to Symmetric CSS Recovery

October 26, 2026

Abstract

For a binary CSS code whose X- and Z-stabilizer row spaces agree, every odd-weight zero-syndrome word is a nontrivial logical operator. Consequently no deterministic syndrome-only decoder can correct every error supported on an odd-cardinality $(a, 0)$ absorbing set. The obstruction applies to arbitrary code length and check presentations, independently of decoder dynamics. We give a general Lean proof, including the actual-check incidence bridge and recovery modulo the entire opposite stabilizer space. We then formulate the remaining even-support existence question for connected, physically inseparable, positive-dimension codes and zero-initialized syndrome-based Gallager-B. A separate one-iteration benchmark implies that question. Neither existence nor nonexistence of an even-support example is established.

1 Result and motivating question

Morris, Pllaha and Kelley ask whether every absorbing set is trapping for their syndrome-based Gallager-B decoder [1], Section VI, arXiv v2, May 10, 2024. A seven-variable simplex matrix answers the unrestricted dynamical question negatively [4]. A separate fifteen-qubit asymmetric CSS construction recovers all 128 errors on a $(7, 0)$ support, with a connected actual decoding graph, one encoded qubit and no physical product cut [5]. These are cited results, with their complete checked proofs preserved separately.

The natural equal-stabilizer-space restriction has an elementary but decisive parity obstruction. We use *weakly self-dual CSS* in the sense of Bravyi, Leemhuis and Terhal [2], Section 3: the binary X- and Z-stabilizer spaces are equal. This is weaker than requiring identical generating matrices and stronger than allowing a coordinate permutation to exchange unequal spaces. It is distinct from the graph-partition notion of a *symmetric stabilizer* in [1], Definition 6.

Question	Source and scope	Answer/evidence
Every absorbing set traps?	Morris–Pllaha–Kelley, v2 Section VI	No, cited simplex counterexample [4].
Can all errors on an odd $(a, 0)$ support recover in equal spaces?	Project-derived restriction; quantum success, not merely syndrome acceptance	No, Theorems 1–2.
Can a suitable even support recover in a connected equal-space code?	Project-derived Question 1, fixed decoder and physical-cut condition	Unresolved here; exact Lean interface and necessary gates.

Theorem 1 (Odd-support obstruction)**.** *Let H_X, H_Z be finite binary check matrices on n physical coordinates. Write $R_X = \operatorname{row} H_X$ and $R_Z = \operatorname{row} H_Z$. Suppose $H_X H_Z^T = 0$ and $R_X = R_Z$. Every vector of R_Z has even weight. If a has odd weight and $H_X a = 0$, then $a \notin R_Z$ and represents a nontrivial logical Z operator. In particular this applies to the indicator of any odd-cardinality $(a, 0)$ absorbing set in the actual H_X graph.*

Theorem 1 is the classical self-orthogonality parity argument applied to the decoding support. The formal contribution is a general proof with actual-matrix, induced-degree and recovery interfaces. The relevant quotient $R^{\perp}/R$ and its bilinear form are classical; see [3], Section 3. The complete proof and source-comparison ledgers are signposted in Appendix A; Appendix B gives the reference chronology.

2 Binary spaces, incidence and the proof

Vectors lie in $\mathbb{F}_2^n$, addition is XOR, and $u \cdot v = \sum_i u_i v_i$ in $\mathbb{F}_2$. Matrices are finite ordered lists of rows; redundant and zero rows are allowed in the general theorem. The row space is the least set containing zero and every actual row and closed under XOR. Thus it is the full stabilizer space, not a selected basis or a restricted submatrix.

For a selected set A with indicator a, the degree of an actual check r into A is $|\operatorname{supp} r \cap A|$. Its parity is $r \cdot a$. Let $b_H(a)$ count actual checks of odd induced degree, counting check vertices as presented. The Lean incidence bridge proves

$$b_H(a) = 0 \quad \Longleftrightarrow \quad Ha = 0.$$

An absorbing support is nonempty and each selected variable has strictly more adjacent even-induced checks than odd-induced checks, as in [1], Definition 5. The parity obstruction needs only oddness and $b_H(a) = 0$, not the strict-majority property.

Proof of Theorem 1. Orthogonality of the actual X and Z rows extends to both row spaces by bilinearity. If $w \in R_Z = R_X$, then $w \cdot w = 0$. But $w \cdot w = |w| \pmod 2$, so w has even weight. An odd a is therefore outside R_Z. If also $H_X a = 0$, its Z operator commutes with every X stabilizer and is not a Z stabilizer. The incidence identity gives the absorbing-set application. □

Theorem 2 (No universal syndrome-only recovery)**.** *Under Theorem 1's hypotheses, fix an odd a with $H_X a = 0$. No deterministic map $D : \mathbb{F}_2^m \to \mathbb{F}_2^n$ can correct both 0 and a modulo R_Z. Hence no such map corrects every error supported on* $\operatorname{supp} a$.

Proof. Both errors produce syndrome zero. If both were corrected, then $D(0) \in R_Z$ and $a + D(0) \in R_Z$, so their sum a would belong to R_Z, contradicting Theorem 1. Both 0 and a are supported on $\operatorname{supp} a$. □

For any decoder that stops at zero syndrome with correction zero, the particular full-support error a fails logically at initialization. This is immediate logical miscorrection, not a proof of persistent syndrome mismatch or message nonconvergence. Quantum recovery and dynamical trapping must remain distinct.

3 Presentations and boundary checks

Proposition 3 (Common presentation and sectors)**.** *If $R_X = R_Z$ and the CSS condition holds, replacing H_Z by the existing H_X preserves the full binary stabilizer group, success modulo Z stabilizers, and closure of the stabilizer space under restriction to any physical cut. It leaves the actual H_X decoding graph and its message updates unchanged. For a common matrix H, a syndrome map that corrects every binary error on A corrects both components of every Pauli error on A when applied separately to the X and Z components.*

Proof. Membership in R_Z can be replaced by membership in R_X. This identifies both factors of the binary stabilizer space $R_X \times R_Z$ with $R_X \times R_X$, including membership of their coordinate restrictions. Orthogonality extends to the common presentation. The decoder uses the unchanged X check list. Finally the two corrected binary residuals belong separately to R, so their pair belongs to $R \times R$. □

This preserves the chosen Z-error decoder; it does not claim that replacing the opposite check presentation preserves a previously different X-error decoder. In the common presentation both components can use the same rule.

The hypotheses have concrete checked guards. On two coordinates the common row $(1, 1)$ has an even zero-syndrome word that is itself a stabilizer: oddness is essential. On six coordinates take $H_X = (111000)$ and $H_Z = (000111)$. The rows commute and swapping the two halves exchanges them, but the odd word 000111 is a Z stabilizer. Thus permutation-exchange symmetry does not imply the theorem's premise. Two distinct generating lists can also span the same space: identical matrices are sufficient, not necessary.

For the seven-coordinate simplex consumer, the common rows are the masks 85, 102, 51, 120, 45, 30, 75 (least-significant-bit coordinate convention). The full support is odd, every induced check has degree four, and each selected variable has four even and zero odd adjacent checks. It is absorbing and zero-syndrome, but logical in the common-matrix CSS code. The exact first-stopping definitions below reject quantum success for this word under both syndrome tests. It also supplies an anticommuting logical pair by pairing its binary vector with itself across the two sectors.

An independent integer-mask implementation checks nine fixed boundary families, including the already published asymmetric example. Under the seven equal-space controls it examines all 167 words, 13 odd zero-syndrome cases and all 1,058 possible shared outputs for those cases. This is finite semantic validation, not a substitute for the arbitrary-length theorem and not an even-support construction search.

4 The remaining existence question

We now freeze a project-derived question rather than attribute it to the source authors. Use one actual binary row list H in both CSS sectors and require $HH^T = 0$. Check vertices are the actual list positions; their ordering affects only syndrome indexing. Distinct rows are required for the existence target, although linear redundancy is allowed.

Use syndrome-based Gallager-B with variable-to-check messages initially zero. A check-to-variable message is its target syndrome bit XOR all other incident incoming messages. Hard output is the strict majority of all incoming check messages; a variable-to-check update uses the strict majority excluding that recipient check. Every tie returns zero. No channel-dependent prior vote is added. At time zero the hard output and all messages are zero. After each full iteration consider two separately stopped runs: the hard syndrome of the hard output, or the outgoing variable-to-check parity at each check. Each run accepts at initialization for syndrome zero, otherwise at its first matching iteration, returning that iteration's hard output. The two runs may stop at different times.

Question 1. *Does there exist a finite common matrix H and nonempty proper $(a, 0)$ absorbing support A such that:*

1. *the actual bipartite Tanner graph is connected and all check rows are distinct;*
2. *there is a pair $x, z \in R^\perp \setminus R$ with $x \cdot z = 1$, witnessing positive logical content;*

3. *for every nontrivial physical coordinate cut C, the binary stabilizer space $R \times R$ is not closed under restriction to C;*

4. *every binary error supported on A reaches a finite first acceptance and is corrected modulo the entire R, under each of the two stopped rules?*

There is no prescribed girth, row/column degree, bounded-degree family, distance, or asymptotic LDPC condition. Four-cycles and linearly redundant checks are allowed. Product separation refers to physical coordinate partitions, not arbitrary entangling Clifford changes of variables. Membership in the full opposite stabilizer span is the quantum success criterion; merely matching a syndrome is insufficient.

Proposition 4 (Necessary gates and benchmark)**.** *Any candidate for Question 1 has even $|A|$. Every zero-syndrome error supported on A must belong to R. Requiring every supported error to succeed at initialization or after one iteration under both tests implies Question 1's recovery condition. Thus the unrestricted support-parity version of Question 1 is equivalent to its even-support version.*

Proof. A zero-syndrome input first accepts at time zero and returns zero, so it succeeds exactly when the input lies in R. Apply this to the indicator of A and use Theorem 1 to get evenness. For a nonzero syndrome, first-iteration acceptance is the first acceptance because initialization did not accept. The stated one-iteration conditions therefore give the respective first-stopped successes. The parity equivalence follows by adding the necessary evenness condition to every candidate. □

No converse from eventual success to one-iteration success is proved or assumed. Failure of a bounded search for the benchmark would leave Question 1 unresolved. Neither question is answered by the present paper. A future construction attempt should first reject supported nonstabilizer zero-syndrome words and then test complete supported-error trajectories; that attempt is outside this artifact.

5 Relation to existing results and limits

Bravyi–Leemhuis–Terhal [2], Section 3, supplies precisely the equal-space/self-orthogonality framework. Their Section 5, Corollary 1 maps any $[[n, k, d]]$ stabilizer code to a $[[4n, 2k, 2d]]$ weakly self-dual CSS code while preserving geometric locality up to constants. Its proofs control code parameters and locality. Applying it here still requires a bridge preserving the selected absorbing support, the actual Gallager-B votes and first stopping, and recovery of every error on that support. We have not established those bridges; the code conversion alone is not a checked solution of Question 1.

Haah–Hastings–Poulin–Wecker [3], Sections 3–4, studies self-orthogonal spaces, logical quotient forms and weakly self-dual code constructions for magic-state distillation. Its normal/hyperbolic distinction concerns the quotient bilinear form, not odd/even cardinality of an absorbing support. These results do not supply the particular stopped Gallager-B recovery certificate required here. We use their algebraic framework as prior context, without claiming the underlying parity fact is new.

Morris–Pllaha–Kelley [1], Theorem 5, already distinguishes logical and degenerate zero-syndrome patterns using the induced code. An induced submatrix's row space is not automatically the entire opposite ambient stabilizer space on a proper support; we explicitly use the latter. Its Definition 3 is dynamical, whereas our obstruction is logical. Section II-B also discusses same-syndrome logical errors; syndrome equality must not be substituted for stabilizer equivalence.

The exact final journal full text remains subscription-restricted in this review. The public journal introduction and accessible arXiv v2 were checked separately; no assertion of identical journal wording is made.

The bounded primary-source and public-code comparison is recorded in the linked ledger. Direct authenticated X search was unavailable. These limits constrain a current-status or priority inference; the precise remaining target is unresolved in this artifact. The unrestricted source question remains answered negatively by the cited counterexample. The odd-support equal-space quantum-recovery restriction is answered negatively here. The remaining even-support existence problem is a separate follow-up.

A Lean interfaces, complete proofs and reproduction

The new proof closure consists of `OddSupport.lean`, `Question.lean`, `Consumers.lean` and the declaration audit. It imports only `Std` and builds cleanly with Lean 4.36.0-rc1. Every one of the 76 theorem declarations has an axiom report restricted to `propext`, `Classical.choice` and `Quot.sound`. No admitted proof or native computation oracle is used. Small Boolean cases and fixed finite consumers use kernel-evaluated `decide`; general implications use structural proofs.

A.1 Proof map and declaration links

The names below are in namespace `CSSOddSupport`, with `Question` or `Consumers` appended for their modules. Complete current and immutable declaration links follow the map.

Written proof	Exact Lean coverage
Incidence/zero syndrome	`dot_intersection_weight`, `zero_b_iff_induced_even`, `zero_b_iff_zero_syndrome`
Theorem 1	`css_spans_commute`, `equal_spans_even_stabilizers`, `odd_zero_syndrome_logical`, `odd_zero_b_logical`
Theorem 2	`cannot_correct_identity_and_odd_support`, `no_universal_supported_recovery`
Zero-stopping application	`zero_stopping_decoder_fails_odd_support`, Question's two `zero_syndrome_*_success_iff`
Proposition 3	`common_matrix_css`, `common_matrix_supported_recovery_iff`, `common_matrix_stabilizer_iff`, `common_matrix_product_cut_iff`, `common_matrix_both_sectors`
Proposition 4	Question's `full_candidate_no_supported_logical`, `full_candidate_support_even`, `full_question_iff_even`, `benchmark_implies_full`
All boundary/consumer assertions	Complete `Consumers.lean` below; independent fixed-family replay in `verify.py`
Question 1	`Structural`, `FullCandidate`, `FullQuestion`; these are definitions, not an existence theorem.

`dot_self_weight`: current, immutable.
`css_spans_commute`: current, immutable.
`odd_zero_b_logical`: current, immutable.
`no_universal_supported_recovery`: current, immutable.
`zero_b_iff_induced_even`: current, immutable.
`common_matrix_css`: current, immutable.
`common_matrix_product_cut_iff`: current, immutable.
`common_matrix_both_sectors`: current, immutable.

`zero_syndrome_hard_success_iff`: current, immutable.
`full_question_iff_even`: current, immutable.
`benchmark_implies_full`: current, immutable.
`seven_no_decoder_corrects_all`: current, immutable.
`permutation_only_spaces_differ`: current, immutable.

The verification manifest records every declaration, source hash and dependency. The comparison ledger and source-question screen retain access limits and question provenance. Running `build.py -lean /path/to/lean` checks a temporary clean import closure and independently replays the fixed certificate.

A.2 Finite binary interface and the general proof

`Word n` is an actual function on the finite physical coordinates, and `Span` is inductively generated by zero, actual rows and XOR. Orthogonality extends by induction. The complete module below includes the parity identity, no-decoder proof, incidence bridge and presentation/sector proofs. Its general theorem allows all finite lengths and arbitrary finite row lists, including empty and degenerate boundaries.

```
import Std

/-! General binary CSS parity obstruction. The physical length and both finite
check lists are arbitrary. No finite enumeration of codes or decoders is used. -/
namespace CSSOddSupport
set_option maxHeartbeats 0

abbrev Word (n : Nat) := Fin n → Bool

def zero : Word n := fun _ => false
def xor (x y : Word n) : Word n := fun i => Bool.xor (x i) (y i)

def dotOn (coords : List α) (x y : α → Bool) : Bool :=
  match coords with
  | [] => false
  | i :: rest => Bool.xor (x i && y i) (dotOn rest x y)

def dot (x y : Word n) : Bool := dotOn (List.finRange n) x y

def weightOn (coords : List α) (x : α → Bool) : Nat := (coords.filter x).length

def weight (x : Word n) : Nat := weightOn (List.finRange n) x

def Odd (x : Word n) : Prop := weight x % 2 = 1

theorem bool_and_xor (a b c : Bool) :
  ((Bool.xor a b) && c) = Bool.xor (a && c) (b && c) := by decide +revert

theorem xor_shuffle (a b c d : Bool) :
  Bool.xor (Bool.xor a b) (Bool.xor c d) =
    Bool.xor (Bool.xor a c) (Bool.xor b d) := by decide +revert

theorem dotOn_comm (coords : List α) (x y : α → Bool) :
  dotOn coords x y = dotOn coords y x := by
  induction coords with
  | nil => rfl
  | cons i rest ih => simp only [dotOn, Bool.and_comm, ih]

theorem dot_comm (x y : Word n) : dot x y = dot y x := dotOn_comm _ _ _

theorem dotOn_zero_left (coords : List α) (x : α → Bool) :
  dotOn coords (fun _ => false) x = false := by
  induction coords with
  | nil => rfl
  | cons i rest ih => simp [dotOn, ih]
```

```
theorem dot_zero_left (x : Word n) : dot zero x = false := dotOn_zero_left _ _

theorem dot_zero_right (x : Word n) : dot x zero = false := by
  rw [dot_comm, dot_zero_left]

theorem dotOn_xor_left (coords : List α) (x y z : α → Bool) :
  dotOn coords (fun i => Bool.xor (x i) (y i)) z =
    Bool.xor (dotOn coords x z) (dotOn coords y z) := by
  induction coords with
  | nil => rfl
  | cons i rest ih =>
    simp only [dotOn, bool_and_xor, ih]
    exact xor_shuffle _ _ _ _

theorem dot_xor_left (x y z : Word n) :
  dot (xor x y) z = Bool.xor (dot x z) (dot y z) := dotOn_xor_left _ _ _ _

theorem dot_xor_right (x y z : Word n) :
  dot x (xor y z) = Bool.xor (dot x y) (dot x z) := by
  rw [dot_comm x, dot_xor_left, dot_comm y x, dot_comm z x]

theorem parity_succ (k : Nat) :
  Bool.xor true (decide (k % 2 = 1)) = decide ((k + 1) % 2 = 1) := by
  have h : k % 2 = 0 ∨ k % 2 = 1 := by omega
  rcases h with h | h <;> simp [Nat.add_mod, h]

theorem dotOn_self_weight (coords : List α) (x : α → Bool) :
  dotOn coords x x = decide (weightOn coords x % 2 = 1) := by
  induction coords with
  | nil => rfl
  | cons i rest ih =>
    rw [dotOn, ih]
    cases hx : x i with
    | false =>
      have hw : weightOn (i :: rest) x = weightOn rest x := by simp [weightOn, hx]
      rw [hw]
      exact Bool.false_xor _
    | true =>
      have hw : weightOn (i :: rest) x = weightOn rest x + 1 := by simp [weightOn, hx]
      rw [hw]
      exact parity_succ (weightOn rest x)

theorem dot_self_weight (x : Word n) :
  dot x x = decide (weight x % 2 = 1) := dotOn_self_weight _ _

theorem odd_dot_self (x : Word n) (h : Odd x) : dot x x = true := by
  rw [dot_self_weight]
  exact decide_eq_true h

-- The least set containing the actual rows and closed under binary addition.
inductive Span (rows : List (Word n)) : Word n → Prop
  | zero : Span rows zero
  | row {r : Word n} : r ∈ rows → Span rows r
  | xor {x y : Word n} : Span rows x → Span rows y → Span rows (xor x y)

theorem span_minimal {rows : List (Word n)} {P : Word n → Prop}
  (hzero : P zero) (hrows : ∀ r, r ∈ rows → P r)
  (hxor : ∀ x y, P x → P y → P (xor x y))
  {w : Word n} (hw : Span rows w) : P w := by
  induction hw with
  | zero => exact hzero
  | row hr => exact hrows _ hr
  | xor hx hy ihx ihy => exact hxor _ _ ihx ihy

def CSS (hx hz : List (Word n)) : Prop :=
  ∀ x, x ∈ hx → ∀ z, z ∈ hz → dot x z = false

def SameSpan (hx hz : List (Word n)) : Prop :=
  ∀ w, Span hx w ↔ Span hz w

theorem row_orthogonal_span {rows : List (Word n)} (x : Word n)
```

```
  (h : ∀ z, z ∈ rows → dot x z = false)
  {z : Word n} (hz : Span rows z) : dot x z = false := by
  induction hz with
  | zero => exact dot_zero_right x
  | row hr => exact h _ hr
  | xor hu hv ihu ihv => rw [dot_xor_right, ihu, ihv]; rfl

theorem css_spans_commute {hx hz : List (Word n)} (hcss : CSS hx hz)
  {x z : Word n} (hX : Span hx x) (hZ : Span hz z) : dot x z = false := by
  induction hX with
  | zero => exact dot_zero_left z
  | row hr => exact row_orthogonal_span _ (hcss _ hr) hZ
  | xor hu hv ihu ihv => rw [dot_xor_left, ihu, ihv]; rfl

theorem equal_spans_self_orthogonal {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz)
  {w : Word n} (hw : Span hz w) : dot w w = false := by
  exact css_spans_commute hcss ((hsame w).mpr hw) hw

theorem equal_spans_even_stabilizers {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz)
  {w : Word n} (hw : Span hz w) : weight w % 2 = 0 := by
  have h := equal_spans_self_orthogonal hcss hsame hw
  rw [dot_self_weight] at h
  have hn : ¬ weight w % 2 = 1 := by simpa using h
  omega

theorem odd_not_stabilizer {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz)
  (a : Word n) (hodd : Odd a) : ¬ Span hz a := by
  intro ha
  have h0 := equal_spans_self_orthogonal hcss hsame ha
  have h1 := odd_dot_self a hodd
  rw [h1] at h0
  contradiction

def ZeroSyndrome (rows : List (Word n)) (a : Word n) : Prop :=
  ∀ r, r ∈ rows → dot r a = false

def LogicalZ (hx hz : List (Word n)) (a : Word n) : Prop :=
  ZeroSyndrome hx a ∧ ¬ Span hz a

def oddChecks (rows : List (Word n)) (a : Word n) : List (Word n) :=
  rows.filter (fun r => dot r a)

def bValue (rows : List (Word n)) (a : Word n) : Nat := (oddChecks rows a).length

theorem zero_b_iff_zero_syndrome (rows : List (Word n)) (a : Word n) :
  bValue rows a = 0 ↔ ZeroSyndrome rows a := by
  simp [bValue, oddChecks, List.filter_eq_nil_iff, ZeroSyndrome]

theorem odd_zero_syndrome_logical {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (hodd : Odd a) (hzero : ZeroSyndrome hx a) : LogicalZ hx hz a := by
  exact ⟨hzero, odd_not_stabilizer hcss hsame a hodd⟩

theorem odd_zero_b_logical {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (hodd : Odd a) (hb : bValue hx a = 0) : LogicalZ hx hz a := by
  exact odd_zero_syndrome_logical hcss hsame a hodd
    ((zero_b_iff_zero_syndrome hx a).mp hb)

theorem identical_rows_same_span (rows : List (Word n)) : SameSpan rows rows := by
  intro w
  rfl

theorem identical_matrix_odd_zero_b {rows : List (Word n)}
  (hcss : CSS rows rows) (a : Word n)
  (hodd : Odd a) (hb : bValue rows a = 0) : LogicalZ rows rows a := by
  exact odd_zero_b_logical hcss (identical_rows_same_span rows) a hodd hb
```

```
def syndrome (rows : List (Word n)) (a : Word n) : List Bool :=
  rows.map (fun r => dot r a)

theorem zero_syndrome_equal (rows : List (Word n)) (a : Word n)
  (h : ZeroSyndrome rows a) : syndrome rows a = syndrome rows zero := by
  apply List.map_congr_left
  intro r hr
  rw [h r hr, dot_zero_right]

theorem xor_zero_left (x : Word n) : xor zero x = x := by
  funext i
  simp [xor, zero]

theorem xor_cancel_right (x y : Word n) : xor (xor x y) y = x := by
  funext i
  change Bool.xor (Bool.xor (x i) (y i)) (y i) = x i
  cases x i <;> cases y i <;> rfl

-- A decoder sees the syndrome only, and success is modulo the entire Z span.
abbrev Decoder (n : Nat) := List Bool → Word n

def Corrects (hx hz : List (Word n)) (decoder : Decoder n) (e : Word n) : Prop :=
  Span hz (xor e (decoder (syndrome hx e)))

theorem cannot_correct_identity_and_odd_support {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (hodd : Odd a) (hzero : ZeroSyndrome hx a) (decoder : Decoder n) :
  ¬ (Corrects hx hz decoder zero ∧ Corrects hx hz decoder a) := by
  intro ⟨h0,ha⟩
  unfold Corrects at h0 ha
  rw [xor_zero_left] at h0
  rw [zero_syndrome_equal hx a hzero] at ha
  have hr := Span.xor ha h0
  rw [xor_cancel_right] at hr
  exact odd_not_stabilizer hcss hsame a hodd hr

def Supported (e a : Word n) : Prop := ∀ i, e i = true → a i = true

def CorrectsSupported (hx hz : List (Word n)) (decoder : Decoder n) (a : Word n) : Prop :=
  ∀ e, Supported e a → Corrects hx hz decoder e

theorem zero_supported (a : Word n) : Supported zero a := by
  intro i h
  contradiction

theorem self_supported (a : Word n) : Supported a a := by
  intro i h
  exact h

theorem no_universal_supported_recovery {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (hodd : Odd a) (hb : bValue hx a = 0) (decoder : Decoder n) :
  ¬ CorrectsSupported hx hz decoder a := by
  intro hall
  exact cannot_correct_identity_and_odd_support hcss hsame a hodd
    ((zero_b_iff_zero_syndrome hx a).mp hb) decoder
    ⟨hall zero (zero_supported a), hall a (self_supported a)⟩

-- Induced check degree is the Hamming weight of the coordinate intersection.
def intersectionWeight (x y : Word n) : Nat := weight (fun i => x i && y i)

theorem dotOn_intersection_self (coords : List α) (x y : α → Bool) :
  dotOn coords x y = dotOn coords (fun i => x i && y i) (fun i => x i && y i) := by
  induction coords with
  | nil => rfl
  | cons i rest ih => simp only [dotOn, Bool.and_self, ih]

theorem dot_intersection_weight (x y : Word n) :
  dot x y = decide (intersectionWeight x y % 2 = 1) := by
  unfold dot
  rw [dotOn_intersection_self, dotOn_self_weight]
```

```
  rfl

theorem dot_false_iff_even_intersection (x y : Word n) :
  dot x y = false ↔ intersectionWeight x y % 2 = 0 := by
  rw [dot_intersection_weight]
  constructor
  · intro h
    have hn : ¬ intersectionWeight x y % 2 = 1 := by simpa using h
    omega
  · intro h
    simp [h]

theorem zero_b_iff_induced_even (rows : List (Word n)) (a : Word n) :
  bValue rows a = 0 ↔ ∀ r, r ∈ rows → intersectionWeight r a % 2 = 0 := by
  rw [zero_b_iff_zero_syndrome]
  constructor
  · intro h r hr
    exact (dot_false_iff_even_intersection r a).mp (h r hr)
  · intro h r hr
    exact (dot_false_iff_even_intersection r a).mpr (h r hr)

def oddNeighborCount (rows : List (Word n)) (a : Word n) (i : Fin n) : Nat :=
  (rows.filter (fun r => r i && dot r a)).length

def evenNeighborCount (rows : List (Word n)) (a : Word n) (i : Fin n) : Nat :=
  (rows.filter (fun r => r i && !(dot r a))).length

def Absorbing (rows : List (Word n)) (a : Word n) : Prop :=
  0 < weight a ∧ ∀ i, a i = true → oddNeighborCount rows a i < evenNeighborCount rows a i

theorem odd_absorbing_no_recovery {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (_habs : Absorbing hx a) (hodd : Odd a) (hb : bValue hx a = 0)
  (decoder : Decoder n) : ¬ CorrectsSupported hx hz decoder a := by
  exact no_universal_supported_recovery hcss hsame a hodd hb decoder

theorem odd_stabilizer_forbids_equal_spaces {hx hz : List (Word n)}
  (hcss : CSS hx hz) (a : Word n) (ha : Span hz a) (hodd : Odd a) :
  ¬ SameSpan hx hz := by
  intro hs
  exact odd_not_stabilizer hcss hs a hodd ha

-- The common-matrix presentation retains the original X decoding rows.
theorem common_matrix_css {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) : CSS hx hx := by
  intro x hxmem z hzmem
  exact css_spans_commute hcss (Span.row hxmem) ((hsame z).mp (Span.row hzmem))

theorem common_matrix_recovery_iff {hx hz : List (Word n)}
  (hsame : SameSpan hx hz) (decoder : Decoder n) (e : Word n) :
  Corrects hx hz decoder e ↔ Corrects hx hx decoder e := by
  exact (hsame _).symm

theorem common_matrix_supported_recovery_iff {hx hz : List (Word n)}
  (hsame : SameSpan hx hz) (decoder : Decoder n) (a : Word n) :
  CorrectsSupported hx hz decoder a ↔ CorrectsSupported hx hx decoder a := by
  constructor
  · intro h e he
    exact (common_matrix_recovery_iff hsame decoder e).mp (h e he)
  · intro h e he
    exact (common_matrix_recovery_iff hsame decoder e).mpr (h e he)

def Stabilizer (hx hz : List (Word n)) (p : Word n × Word n) : Prop :=
  Span hx p.1 ∧ Span hz p.2

def restrict (w cut : Word n) : Word n := fun i => w i && cut i

def SplitsAcross (hx hz : List (Word n)) (cut : Word n) : Prop :=
  ∀ p, Stabilizer hx hz p → Stabilizer hx hz (restrict p.1 cut, restrict p.2 cut)

theorem common_matrix_stabilizer_iff {hx hz : List (Word n)}
```

```
  (hsame : SameSpan hx hz) (p : Word n × Word n) :
  Stabilizer hx hz p ↔ Stabilizer hx hx p := by
  exact and_congr Iff.rfl (hsame p.2).symm

theorem common_matrix_product_cut_iff {hx hz : List (Word n)}
  (hsame : SameSpan hx hz) (cut : Word n) :
  SplitsAcross hx hz cut ↔ SplitsAcross hx hx cut := by
  constructor
  · intro h p hp
    exact (common_matrix_stabilizer_iff hsame _).mp
      (h p ((common_matrix_stabilizer_iff hsame p).mpr hp))
  · intro h p hp
    exact (common_matrix_stabilizer_iff hsame _).mpr
      (h p ((common_matrix_stabilizer_iff hsame p).mp hp))

-- For a common matrix, the same syndrome recovery map can decode both components.
def CorrectsPauli (rows : List (Word n)) (decoder : Decoder n) (e : Word n × Word n) : Prop :=
  Stabilizer rows rows
    (xor e.1 (decoder (syndrome rows e.1)), xor e.2 (decoder (syndrome rows e.2)))

theorem common_matrix_both_sectors {rows : List (Word n)} (decoder : Decoder n)
  (a : Word n) (h : CorrectsSupported rows rows decoder a)
  (e : Word n × Word n) (hx : Supported e.1 a) (hz : Supported e.2 a) :
  CorrectsPauli rows decoder e := by
  exact ⟨h e.1 hx, h e.2 hz⟩

theorem successful_zero_b_support_even {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (hb : bValue hx a = 0) (decoder : Decoder n)
  (h : CorrectsSupported hx hz decoder a) : weight a % 2 = 0 := by
  have hn : ¬ Odd a := by
    intro ho
    exact no_universal_supported_recovery hcss hsame a ho hb decoder h
  unfold Odd at hn
  omega

theorem xor_zero_right (x : Word n) : xor x zero = x := by
  funext i
  exact Bool.xor_false _

theorem css_iff_check_all (hx hz : List (Word n)) :
  CSS hx hz ↔ (hx.all (fun x => hz.all (fun z => !(dot x z)))) = true := by
  simp [CSS, List.all_eq_true]

theorem zero_stopping_decoder_fails_odd_support {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (hodd : Odd a) (hzero : ZeroSyndrome hx a) (decoder : Decoder n)
  (hstop : decoder (syndrome hx zero) = zero) : ¬ Corrects hx hz decoder a := by
  intro hc
  unfold Corrects at hc
  rw [zero_syndrome_equal hx a hzero, hstop, xor_zero_right] at hc
  exact odd_not_stabilizer hcss hsame a hodd hc

end CSSOddSupport
```

A.3 Actual messages and first stopping

Check indices enumerate the actual row list. Nonedges are set to zero; majority is strict, with zero on ties. The trajectory is defined unhalted only to locate the first accepting time. `Success` insists on that first time and its hard output. `FullQuestion` and `OneStepQuestion` remain unproved propositions; the only existence implication proved is from the benchmark to the full question.

```
import OddSupport

/-! Exact project-derived existence questions. No existence or nonexistence
proof for even supports is asserted. The actual common check list is used in
all graph incidences, messages and stabilizer spaces. -/
```

```
namespace CSSOddSupport.Question
set_option maxHeartbeats 0

abbrev Check (rows : List (Word n)) := Fin rows.length
abbrev Msg (rows : List (Word n)) := Check rows → Fin n → Bool

def rowAt (rows : List (Word n)) (c : Check rows) : Word n := rows[c.val]
def adj (rows : List (Word n)) (c : Check rows) (v : Fin n) : Bool := rowAt rows c v

def parity (bs : List Bool) : Bool := bs.foldr Bool.xor false

def majority (bs : List Bool) : Bool := decide (bs.length < 2 * bs.countP id)

def checkUpdate (rows : List (Word n)) (s : List Bool) (x : Msg rows) : Msg rows :=
  fun c v => adj rows c v && Bool.xor (s[c.val]?.getD false)
    (parity ((List.finRange n).map (fun u => adj rows c u && (u != v) && x c u)))

def hard (rows : List (Word n)) (y : Msg rows) : Word n :=
  fun v => majority (((List.finRange rows.length).filter (fun c => adj rows c v)).map (fun c => y c v))

def variableUpdate (rows : List (Word n)) (y : Msg rows) : Msg rows :=
  fun c v => adj rows c v && majority
    (((List.finRange rows.length).filter (fun d => adj rows d v && (d != c))).map (fun d => y d v))

def edgeSyndrome (rows : List (Word n)) (x : Msg rows) : List Bool :=
  (List.finRange rows.length).map (fun c =>
    parity ((List.finRange n).map (fun v => adj rows c v && x c v)))

def variables (rows : List (Word n)) (s : List Bool) : Nat → Msg rows
  | 0 => fun _ _ => false
  | t+1 => variableUpdate rows (checkUpdate rows s (variables rows s t))

def output (rows : List (Word n)) (s : List Bool) : Nat → Word n
  | 0 => zero
  | t+1 => hard rows (checkUpdate rows s (variables rows s t))

def hardAccept (rows : List (Word n)) (s : List Bool) : Nat → Prop
  | 0 => s = syndrome rows zero
  | t+1 => syndrome rows (output rows s (t+1)) = s

def edgeAccept (rows : List (Word n)) (s : List Bool) : Nat → Prop
  | 0 => s = syndrome rows zero
  | t+1 => edgeSyndrome rows (variables rows s (t+1)) = s

def FirstAccept (accept : Nat → Prop) (t : Nat) : Prop :=
  accept t ∧ ∀ u, u < t → ¬ accept u

def Success (rows : List (Word n)) (accept : List Bool → Nat → Prop) (e : Word n) : Prop :=
  ∃ t, FirstAccept (accept (syndrome rows e)) t ∧
    Span rows (xor e (output rows (syndrome rows e) t))

def HardSuccess (rows : List (Word n)) (e : Word n) : Prop := Success rows (hardAccept rows) e
def EdgeSuccess (rows : List (Word n)) (e : Word n) : Prop := Success rows (edgeAccept rows) e

def OneStepSuccess (rows : List (Word n)) (e : Word n) : Prop :=
  let s := syndrome rows e
  if s = syndrome rows zero then Span rows e else
    syndrome rows (output rows s 1) = s ∧ edgeSyndrome rows (variables rows s 1) = s ∧
      Span rows (xor e (output rows s 1))

abbrev Node (n : Nat) (rows : List (Word n)) := Sum (Fin n) (Check rows)

def Edge (rows : List (Word n)) : Node n rows → Node n rows → Prop
  | .inl v, .inr c => adj rows c v = true
  | .inr c, .inl v => adj rows c v = true
  | _, _ => False

inductive Walk (rows : List (Word n)) : Node n rows → Node n rows → Prop
  | nil (u : Node n rows) : Walk rows u u
  | cons {u v w : Node n rows} : Edge rows u v → Walk rows v w → Walk rows u w
```

```
def Connected (rows : List (Word n)) : Prop := ∀ u v : Node n rows, Walk rows u v

def LogicalPair (rows : List (Word n)) : Prop :=
  ∃ x z : Word n, LogicalZ rows rows x ∧ LogicalZ rows rows z ∧ dot x z = true

def ones : Word n := fun _ => true

structure Structural (rows : List (Word n)) (a : Word n) : Prop where
  css : CSS rows rows
  distinctChecks : rows.Pairwise (fun x y => x ≠ y)
  absorbing : Absorbing rows a
  zeroOddChecks : bValue rows a = 0
  properSupport : weight a < n
  positiveLogical : LogicalPair rows
  connected : Connected rows
  noProductCut : ∀ cut : Word n, cut ≠ zero → cut ≠ ones → ¬ SplitsAcross rows rows cut

def FullCandidate (rows : List (Word n)) (a : Word n) : Prop :=
  Structural rows a ∧ ∀ e, Supported e a → HardSuccess rows e ∧ EdgeSuccess rows e

def OneStepCandidate (rows : List (Word n)) (a : Word n) : Prop :=
  Structural rows a ∧ ∀ e, Supported e a → OneStepSuccess rows e

-- Primary question and the explicitly stronger finite verification benchmark.
def FullQuestion : Prop := ∃ n, ∃ rows : List (Word n), ∃ a : Word n, FullCandidate rows a

def EvenFullQuestion : Prop := ∃ n, ∃ rows : List (Word n), ∃ a : Word n,
  FullCandidate rows a ∧ weight a % 2 = 0

def OneStepQuestion : Prop := ∃ n, ∃ rows : List (Word n), ∃ a : Word n, OneStepCandidate rows a

theorem first_accept_at_zero {accept : Nat → Prop} {t : Nat}
  (hzero : accept 0) (h : FirstAccept accept t) : t = 0 := by
  by_cases ht : t = 0
  · exact ht
  · have hlt : 0 < t := by omega
    exact False.elim (h.2 0 hlt hzero)

theorem zero_syndrome_hard_success_iff (rows : List (Word n)) (a : Word n)
  (hzero : ZeroSyndrome rows a) : HardSuccess rows a ↔ Span rows a := by
  have hs := zero_syndrome_equal rows a hzero
  constructor
  · rintro ⟨t,ht,hr⟩
    have h0 : hardAccept rows (syndrome rows a) 0 := hs
    have ht0 := first_accept_at_zero h0 ht
    simpa [ht0, output, xor_zero_right] using hr
  · intro ha
    refine ⟨0,⟨hs,?_⟩,?_⟩
    · intro u hu
      omega
    · simpa [output, xor_zero_right] using ha

theorem zero_syndrome_edge_success_iff (rows : List (Word n)) (a : Word n)
  (hzero : ZeroSyndrome rows a) : EdgeSuccess rows a ↔ Span rows a := by
  have hs := zero_syndrome_equal rows a hzero
  constructor
  · rintro ⟨t,ht,hr⟩
    have h0 : edgeAccept rows (syndrome rows a) 0 := hs
    have ht0 := first_accept_at_zero h0 ht
    simpa [ht0, output, xor_zero_right] using hr
  · intro ha
    refine ⟨0,⟨hs,?_⟩,?_⟩
    · intro u hu
      omega
    · simpa [output, xor_zero_right] using ha

theorem one_step_hard_success (rows : List (Word n)) (a : Word n)
  (h : OneStepSuccess rows a) : HardSuccess rows a := by
  unfold OneStepSuccess at h
  dsimp only at h
  by_cases hs : syndrome rows a = syndrome rows zero
```

```
  · have ha : Span rows a := by simpa [hs] using h
    refine ⟨0,⟨hs,?_⟩,?_⟩
    · intro u hu
      omega
    · simpa [output, xor_zero_right] using ha
  · have ha := h
    simp only [hs, ite_false] at ha
    refine ⟨1,⟨ha.1,?_⟩,ha.2.2⟩
    intro u hu
    have hu0 : u = 0 := by omega
    subst u
    exact hs

theorem one_step_edge_success (rows : List (Word n)) (a : Word n)
  (h : OneStepSuccess rows a) : EdgeSuccess rows a := by
  unfold OneStepSuccess at h
  dsimp only at h
  by_cases hs : syndrome rows a = syndrome rows zero
  · have ha : Span rows a := by simpa [hs] using h
    refine ⟨0,⟨hs,?_⟩,?_⟩
    · intro u hu
      omega
    · simpa [output, xor_zero_right] using ha
  · have ha := h
    simp only [hs, ite_false] at ha
    refine ⟨1,⟨ha.2.1,?_⟩,ha.2.2⟩
    intro u hu
    have hu0 : u = 0 := by omega
    subst u
    exact hs

theorem one_step_candidate_full {rows : List (Word n)} {a : Word n}
  (h : OneStepCandidate rows a) : FullCandidate rows a := by
  refine ⟨h.1,?_⟩
  intro e he
  exact ⟨one_step_hard_success rows e (h.2 e he), one_step_edge_success rows e (h.2 e he)⟩

def NoSupportedLogical (rows : List (Word n)) (a : Word n) : Prop :=
  ∀ e, Supported e a → ZeroSyndrome rows e → Span rows e

theorem full_candidate_no_supported_logical {rows : List (Word n)} {a : Word n}
  (h : FullCandidate rows a) : NoSupportedLogical rows a := by
  intro e he hz
  exact (zero_syndrome_hard_success_iff rows e hz).mp (h.2 e he).1

theorem full_candidate_support_even {rows : List (Word n)} {a : Word n}
  (h : FullCandidate rows a) : weight a % 2 = 0 := by
  have hz := (zero_b_iff_zero_syndrome rows a).mp h.1.zeroOddChecks
  have ha := (zero_syndrome_hard_success_iff rows a hz).mp (h.2 a (self_supported a)).1
  exact equal_spans_even_stabilizers h.1.css (identical_rows_same_span rows) ha

theorem full_question_iff_even : FullQuestion ↔ EvenFullQuestion := by
  constructor
  · rintro ⟨n,rows,a,h⟩
    exact ⟨n,rows,a,h,full_candidate_support_even h⟩
  · rintro ⟨n,rows,a,h,_⟩
    exact ⟨n,rows,a,h⟩

theorem benchmark_implies_full : OneStepQuestion → FullQuestion := by
  rintro ⟨n,rows,a,h⟩
  exact ⟨n,rows,a,one_step_candidate_full h⟩

end CSSOddSupport.Question
```

A.4 Fixed consumers and hypothesis guards

These controls test oddness, actual selected degrees, empty-coordinate boundaries, row-space equality versus literal equality, and permutation-only symmetry. The seven-coordinate consumer is a

logical-failure control, not a successful even-support construction.

```
import Question

/-! Small independent consumers of the general theorem. These do not search
for an even-support construction. -/
namespace CSSOddSupport.Consumers

def maskWord (n k : Nat) : Word n := fun i => (BitVec.ofNat n k).getLsbD i.val

def simplex : List (Word 7) := [85,102,51,120,45,30,75].map (maskWord 7)
def fullSeven : Word 7 := maskWord 7 127

theorem simplex_css : CSS simplex simplex := by
  apply (css_iff_check_all _ _).mpr
  decide

theorem seven_is_odd : Odd fullSeven := by
  unfold Odd
  decide

theorem seven_is_absorbing : Absorbing simplex fullSeven := by
  unfold Absorbing
  decide

theorem seven_has_zero_odd_checks : bValue simplex fullSeven = 0 := by decide

theorem seven_is_logical : LogicalZ simplex simplex fullSeven := by
  exact identical_matrix_odd_zero_b simplex_css fullSeven seven_is_odd seven_has_zero_odd_checks

theorem seven_no_decoder_corrects_all (decoder : Decoder 7) :
  ¬ CorrectsSupported simplex simplex decoder fullSeven := by
  exact no_universal_supported_recovery simplex_css (identical_rows_same_span simplex)
    fullSeven seven_is_odd seven_has_zero_odd_checks decoder

theorem seven_hard_stopping_logical_failure : ¬ Question.HardSuccess simplex fullSeven := by
  intro h
  have hz := (zero_b_iff_zero_syndrome simplex fullSeven).mp seven_has_zero_odd_checks
  exact seven_is_logical.2 ((Question.zero_syndrome_hard_success_iff simplex fullSeven hz).mp h)

theorem seven_edge_stopping_logical_failure : ¬ Question.EdgeSuccess simplex fullSeven := by
  intro h
  have hz := (zero_b_iff_zero_syndrome simplex fullSeven).mp seven_has_zero_odd_checks
  exact seven_is_logical.2 ((Question.zero_syndrome_edge_success_iff simplex fullSeven hz).mp h)

theorem seven_logical_pair : LogicalZ simplex simplex fullSeven ∧
  LogicalZ simplex simplex fullSeven ∧ dot fullSeven fullSeven = true := by
  exact ⟨seven_is_logical,seven_is_logical,odd_dot_self fullSeven seven_is_odd⟩

-- Empty physical length satisfies no odd premise; this is not an excluded hidden case.
theorem zero_length_not_odd (a : Word 0) : ¬ Odd a := by
  simp [Odd, weight, weightOn, List.finRange]

-- Equality of spans need not mean equality of displayed check lists.
def pairRow : Word 2 := maskWord 2 3

theorem one_pair_row_css : CSS [pairRow] [pairRow] := by
  apply (css_iff_check_all _ _).mpr
  decide

theorem even_zero_syndrome_stabilizer :
  ZeroSyndrome [pairRow] pairRow ∧ Span [pairRow] pairRow ∧ weight pairRow % 2 = 0 := by
  refine ⟨?_, Span.row (by simp), ?_⟩
  · unfold ZeroSyndrome
    intro r hr
    simp only [List.mem_singleton] at hr
    subst r
    decide
  · decide

theorem duplicate_row_same_span : SameSpan [pairRow] [pairRow,pairRow] := by
```

```
  intro w
  constructor
  · intro h
    exact span_minimal Span.zero (by intro r hr; exact Span.row (by simpa using hr))
      (by intro _ _ hx hy; exact Span.xor hx hy) h
  · intro h
    exact span_minimal Span.zero (by intro r hr; exact Span.row (by simpa using hr))
      (by intro _ _ hx hy; exact Span.xor hx hy) h

theorem duplicate_lists_differ : ([pairRow] : List (Word 2)) ≠ [pairRow,pairRow] := by
  intro h
  have := congrArg List.length h
  contradiction

-- A permutation exchanging X/Z spaces is a strictly different hypothesis.
def oddLeft : Word 6 := maskWord 6 7
def oddRight : Word 6 := maskWord 6 56

def halfSwap (i : Fin 6) : Fin 6 := ⟨(i.val+3)%6,by omega⟩
def permuteHalves (w : Word 6) : Word 6 := fun i => w (halfSwap i)

theorem permutation_only_css : CSS [oddLeft] [oddRight] := by
  apply (css_iff_check_all _ _).mpr
  decide

theorem halves_exchange_rows : permuteHalves oddLeft = oddRight ∧
  permuteHalves oddRight = oddLeft := by
  constructor
  · funext i
    decide +revert
  · funext i
    decide +revert

theorem permutation_only_has_odd_stabilizer : Odd oddRight ∧ Span [oddRight] oddRight := by
  constructor
  · unfold Odd
    decide
  · exact Span.row (by simp)

theorem permutation_only_spaces_differ : ¬ SameSpan [oddLeft] [oddRight] := by
  exact odd_stabilizer_forbids_equal_spaces permutation_only_css oddRight
    permutation_only_has_odd_stabilizer.2 permutation_only_has_odd_stabilizer.1

end CSSOddSupport.Consumers
```

B Reference chronology

The manuscript date is October 26, 2026. Day counts below are exact differences from the explicitly dated preprint events. They measure source age, not how long a question was unsolved. The repository papers are mathematical predecessors cited at fixed revisions, not evidence of an external announcement date.

Source	Earliest preprint	Inspected milestone	Days since earliest
Bravyi et al.	2010-04-21	v2 April 27, 2010; NJP 2010	6032
Haah et al.	2017-03-22	v3 Sept. 29, 2017; Quantum Oct. 3, 2017	3505
Morris et al.	2023-07-26	v2 May 10, 2024; public journal intro 2026	1188

The BLT journal day is not asserted here; the year is sufficient for the inspected preprint comparison. The P09 final journal full-text access gap is retained in the comparison ledger. Earlier repository constructions are referenced at their immutable publication and proof commits in the bibliography.

References

[1] K. D. Morris, T. Pllaha and C. A. Kelley, *Absorbing Sets in Quantum LDPC Codes*, arXiv:2307.14532v2 (May 10, 2024). Journal metadata: doi:10.3934/amc.2026023 (2026).

[2] S. Bravyi, B. Leemhuis and B. M. Terhal, *Majorana Fermion Codes*, arXiv:1004.3791v2 (April 27, 2010), New J. Phys. 12, 083039 (2010).

[3] J. Haah, M. B. Hastings, D. Poulin and D. Wecker, *Magic State Distillation with Low Space Overhead and Optimal Asymptotic Input Count*, arXiv:1703.07847v3 (September 29, 2017), Quantum 1, 31 (October 3, 2017).

[4] *An Absorbing Set Without Gallager-B Trapping*, paper and checked artifact, vibe-math (2026).

[5] *A Connected CSS Absorbing Set Without Gallager-B Trapping*, paper; immutable checked proof, vibe-math (2026).